Cài đặt Hadoop 3.4.3 trên Docker

> **Cài đặt Hadoop Eco System trên môi trường Docker.** Giữ nguyên tên `minhchau-master`, `minhchau-slave1`, user `hadoopminhchau`.

Mục lục

1. Docker thay thế những bước nào của tài liệu gốc
2. Kiến trúc: cái gì chạy ở đâu
3. Chuẩn bị
4. Cấu trúc thư mục dự án
5. Phần A — Single-node (pseudo-distributed)
6. Phần B — Cluster 1 master + 2 slave
7. Giải thích các file cấu hình
8. Xử lý lỗi thường gặp

---

1. Docker thay thế những bước nào

Khoảng một nửa tài liệu gốc là công việc quản trị hệ điều hành. Docker làm giúp bạn hết:

Mục trong tài liệu gốc	Trên Docker
1. Thiết lập IP tĩnh (`netplan`)	**Bỏ.**
4.2 Sửa `/etc/hosts`	**Bỏ.**
4.3/4.4 Đặt `/etc/hostname`	Khai bằng `hostname:` trong `docker-compose.yml`.

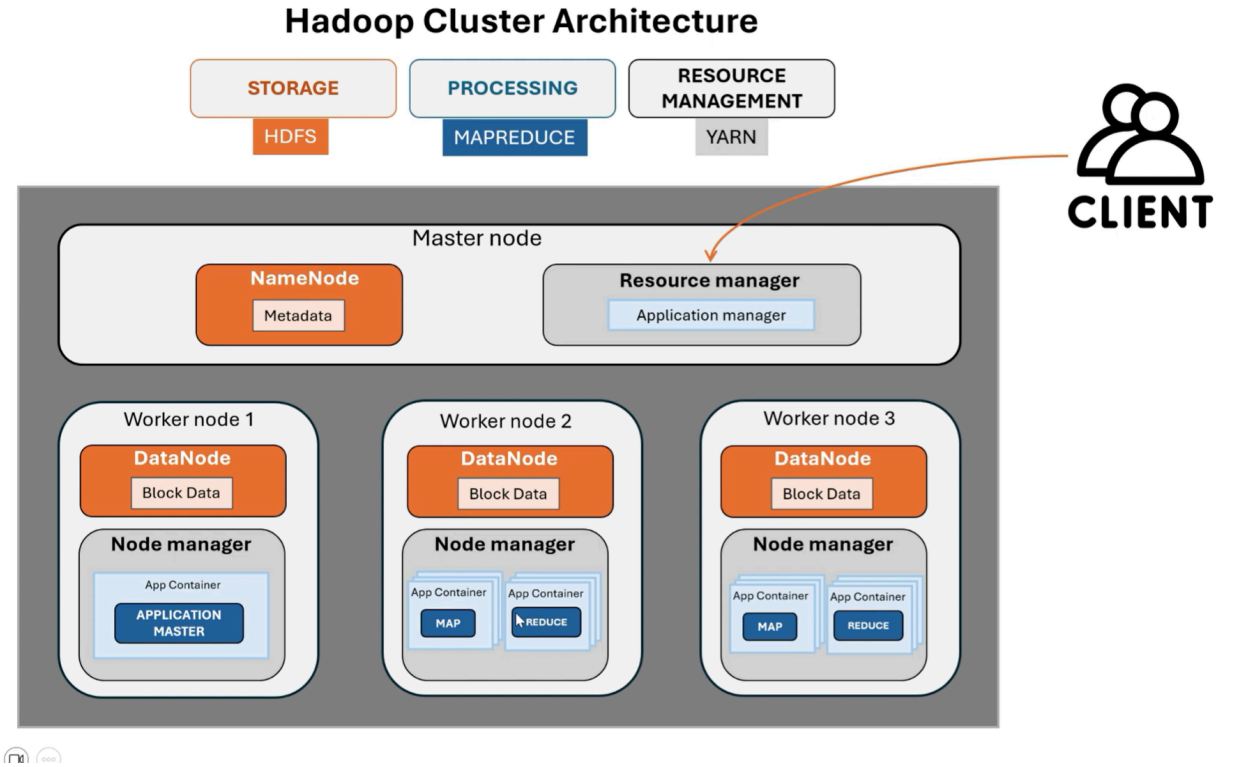
Mục trong tài liệu gốc	Trên Docker
2, 3 Cài JDK, SSH	Viết trong `Dockerfile`, build 1 lần dùng mãi.
5 Tạo user `hadoopminhchau`	Viết trong `Dockerfile`.
6 Tải & giải nén Hadoop	Viết trong `Dockerfile`.
7.1 Sửa `~/.bashrc`	Dùng `ENV` trong `Dockerfile` (+ ghi thêm vào `.bashrc`).
9. Clone máy ảo ra slave1	**Bỏ.** Thêm 1 service trong compose là xong.
10. Trao đổi SSH key	Sinh sẵn trong image. Thực ra Docker **không cần** SSH (xem §6.4).
11–13 Format, start, test	**Giữ nguyên**, chỉ là chạy bên trong container.

---

2. Kiến trúc

Hadoop có 2 hệ thống độc lập, mỗi hệ thống theo mô hình **1 master – nhiều slave**:

		HDFS (lưu trữ)	YARN (tính toán)
minhchau-master	→	NameNode SecondaryNameNode (giữ metadata: file nào, mấy block, ở node nào)	ResourceManager JobHistoryServer (nhận job, chia tài nguyên)
minhchau-slave1	→	DataNode	NodeManager
minhchau-slave2	→	DataNode (giữ block dữ liệu thật trên đĩa)	NodeManager (chạy container tính toán thật)



HDFS – tầng lưu trữ

- **NameNode**: là **master node của HDFS**, quản lý **metadata của filesystem**, không lưu dữ liệu file thực tế. Metadata gồm namespace, thư mục, tên file, quyền truy cập và **mapping file → block → DataNode**.
  → NameNode chết **không đồng nghĩa mất dữ liệu**, vì block vẫn nằm trên DataNode; nhưng cluster có thể không truy cập/điều phối HDFS bình thường nếu không có cơ chế HA.
- **Secondary NameNode**: **không phải standby NameNode và không phải backup NameNode**. Nó định kỳ thực hiện **checkpoint**, tức là merge `fsimage` và `edits` để tạo ra một `fsimage` mới, giúp giảm kích thước `edits` và hỗ trợ khởi động/recovery NameNode nhanh hơn.
- **DataNode**: là node **lưu trữ dữ liệu thực tế dưới dạng HDFS blocks**, đồng thời phục vụ việc đọc/ghi block và thực hiện các tác vụ replication theo yêu cầu của NameNode.
- **Block replication**: HDFS mặc định có **replication factor = 3**, nghĩa là mỗi block thường có **3 replicas** được phân bố trên các DataNode khác nhau, theo chính sách placement của HDFS.
  → Không nên nói đơn giản là "nhân bản sang 3 node khác nhau", vì cách placement còn phụ thuộc topology/rack awareness.

YARN – tầng quản lý tài nguyên và thực thi

- **ResourceManager (RM)**: là **master service của YARN**, chịu trách nhiệm **quản lý và phân bổ tài nguyên cluster** như CPU và memory cho các application. Nó gồm các chức năng chính như **ApplicationsManager** và **Scheduler**.
- **NodeManager (NM)**: là **agent chạy trên từng worker node**, chịu trách nhiệm quản lý tài nguyên và **container** trên node đó; khởi chạy, giám sát và quản lý vòng đời các container theo yêu cầu của ResourceManager/ApplicationMaster.

- **ApplicationMaster (AM)**: đây là thành phần bạn đang thiếu nếu muốn hiểu YARN đầy đủ. **Mỗi application thường có một ApplicationMaster**, chịu trách nhiệm điều phối việc thực thi application, thương lượng resource với ResourceManager và phối hợp các task/container trên NodeManager.

Port cần nhớ:

Cổng	**Dịch vụ**
9000	`fs.defaultFS` — client nói chuyện với NameNode
9870	NameNode Web UI
9864	DataNode Web UI
8088	ResourceManager Web UI
8042	NodeManager Web UI
19888	JobHistory Web UI

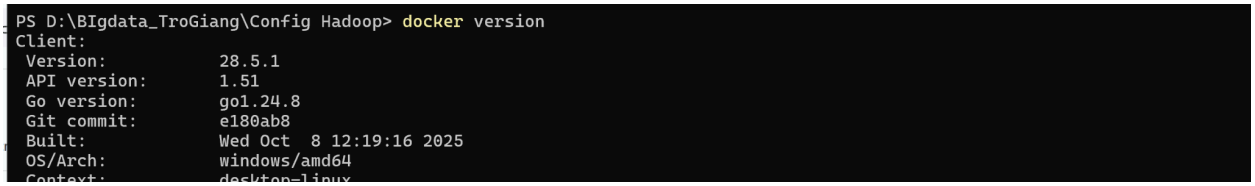
---

3. Chuẩn bị

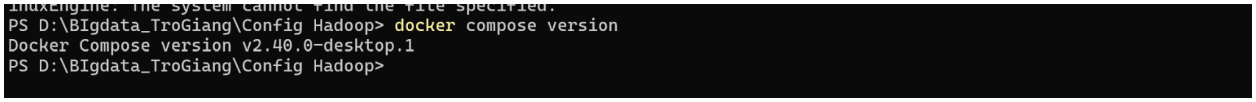
- Docker Desktop (Windows/macOS) hoặc Docker Engine + plugin `compose` (Linux).
- Cấp cho Docker tối thiểu **6 GB RAM** (Docker Desktop → Settings → Resources). Cluster 3 container + JVM khá nặng.
- Khoảng 5 GB đĩa trống.

Kiểm tra:

docker --version



docker compose version



---

4. Cấu trúc thư mục dự án

hadoop-docker/

├── Dockerfile # cách dựng "máy ảo" Hadoop

├── entrypoint.sh # quyết định container đóng vai master hay slave

├── test.sh # bài test WordCount (mục 13 tài liệu)

├── docker-compose.single.yml # Phần A: 1 container

├── docker-compose.cluster.yml # Phần B: 3 container

└── config/

├── common/ # giống nhau trên MỌI node

│ ├── core-site.xml # mục 7.3

```
│   ├── yarn-site.xml          #  mục 7.5
│   └── mapred-site.xml        #  mục 7.6
├── single/                    # chỉ dùng cho Phần A
│   ├── hdfs-site.xml          #  replication = 1
│   └── workers                #  minhchau-master
└── cluster/                   # chỉ dùng cho Phần B
    ├── hdfs-site.xml          #  replication = 2
    └── workers                #  minhchau-slave1, minhchau-slave2
```

Thư mục `config/` được **mount** vào container, nên bạn sửa file XML trên máy thật rồi `docker compose restart` là cấu hình mới có hiệu lực — **không phải build lại image**. Đây là lý do nên tách config ra ngoài thay vì `COPY` vào image.

Cách thực hiện nhanh:

1. **Linux / macOS / Git Bash / WSL — dán nguyên khối:**

```
mkdir -p hadoop-docker/config/{common,single,cluster} && cd hadoop-docker

touch Dockerfile entrypoint.sh test.sh \
    docker-compose.single.yml docker-compose.cluster.yml \
    config/common/{core-site.xml,yarn-site.xml,mapred-site.xml} \
    config/single/{hdfs-site.xml,workers} \
    config/cluster/{hdfs-site.xml,workers}
```

chmod +x entrypoint.sh test.sh

2. **Windows PowerShell:**

mkdir hadoop-docker\config\common,
hadoop-docker\config\single, hadoop-docker\config\cluster
-Force
cd hadoop-docker

$f = @(
  'Dockerfile','entrypoint.sh','test.sh',
  'docker-compose.single.yml','docker-compose.cluster.yml',

'config\common\core-site.xml','config\common\yarn-site.xml','co
nfig\common\mapred-site.xml',
  'config\single\hdfs-site.xml','config\single\workers',
  'config\cluster\hdfs-site.xml','config\cluster\workers'
)
$f | ForEach-Object { New-Item $_ -ItemType File -Force |
Out-Null }

```
PS D:\BIgdata_TroGiang\Config Hadoop> cd hadoop-docker
PS D:\BIgdata_TroGiang\Config Hadoop\hadoop-docker>
PS D:\BIgdata_TroGiang\Config Hadoop\hadoop-docker> $f = @(
>>   'Dockerfile','entrypoint.sh','test.sh',
>>   'docker-compose.single.yml','docker-compose.cluster.yml',
>>   'config\common\core-site.xml','config\common\yarn-site.xml','config\common\mapred-site.xml',
>>   'config\single\hdfs-site.xml','config\single\workers',
>>   'config\cluster\hdfs-site.xml','config\cluster\workers'
>> )
PS D:\BIgdata_TroGiang\Config Hadoop\hadoop-docker> $f | ForEach-Object { New-Item $_ -ItemType File -Force | Out-Null }

PS D:\BIgdata_TroGiang\Config Hadoop\hadoop-docker>
```

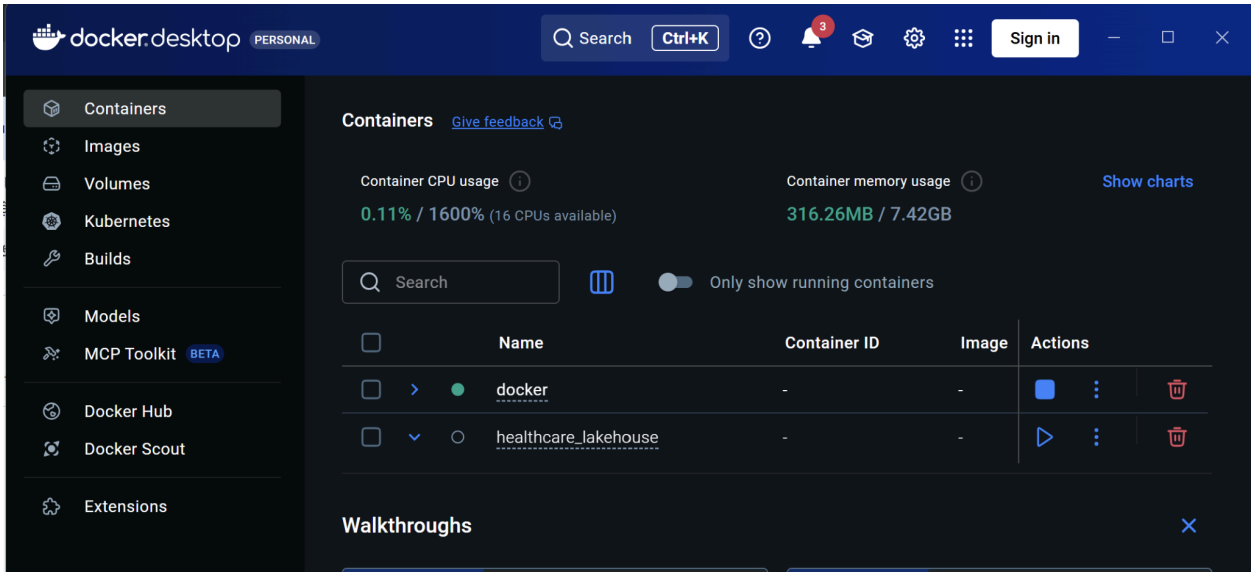
---

5. Phần A — Single-node (pseudo-distributed)

Một container chạy tất cả daemon. Đây là bản "thu nhỏ" để bạn làm quen trước.

5.1 Build và khởi động

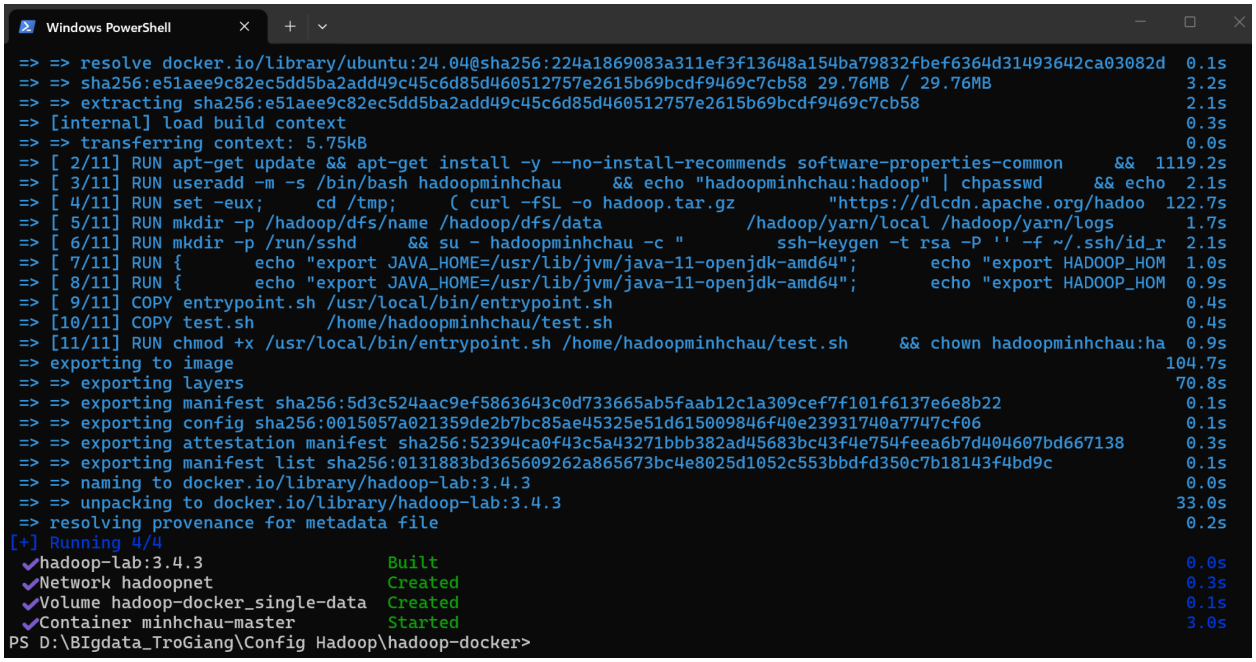
Đầu tiên mở docker desktop trước



Chạy container của single node

docker compose -f docker-compose.single.yml up -d --build

Lần đầu build mất 5–10 phút (tải ~1 GB Hadoop + JDK). Những lần sau dùng cache, chỉ vài giây.



5.2 Kiểm tra các daemon (mục 12 tài liệu)

docker exec -u hadoopminhchau minhchau-master jps

Phải thấy đủ 6 dòng:

NameNode

SecondaryNameNode

DataNode

ResourceManager

NodeManager

JobHistoryServer

```
 ✔Container minhchau-master          Started                                    3.0s
PS D:\BIgdata_TroGiang\Config Hadoop\hadoop-docker> docker exec -u hadoopminhchau minhchau-master jps
448 NodeManager
256 ResourceManager
1012 JobHistoryServer
374 DataNode
1110 Jps
171 SecondaryNameNode
109 NameNode
PS D:\BIgdata_TroGiang\Config Hadoop\hadoop-docker>
```

5.3 Kiểm tra HDFS

docker exec -u hadoopminhchau minhchau-master hdfs dfsadmin -report

Dòng `Live datanodes (1)` là đạt.

```
PS D:\BIgdata_TroGiang\Config Hadoop\hadoop-docker> docker exec -u hadoopminhchau minhchau-master hdfs dfsadmin -report
Configured Capacity: 1081101176832 (1006.85 GB)
Present Capacity: 1010669875200 (941.26 GB)
DFS Remaining: 1010669850624 (941.26 GB)
DFS Used: 24576 (24 KB)
DFS Used%: 0.00%
Replicated Blocks:
        Under replicated blocks: 0
        Blocks with corrupt replicas: 0
        Missing blocks: 0
        Missing blocks (with replication factor 1): 0
        Low redundancy blocks with highest priority to recover: 0
        Pending deletion blocks: 0
Erasure Coded Block Groups:
        Low redundancy block groups: 0
        Block groups with corrupt internal blocks: 0
        Missing block groups: 0
        Low redundancy blocks with highest priority to recover: 0
        Pending deletion blocks: 0

-------------------------------------------------
Live datanodes (1):

Name: 172.20.0.2:9866 (minhchau-master)
Hostname: minhchau-master
Decommission Status : Normal
Configured Capacity: 1081101176832 (1006.85 GB)
DFS Used: 24576 (24 KB)
```

5.4 Mở Web UI

- HDFS: http://localhost:9870
- YARN: http://localhost:8088
- JobHistory: http://localhost:19888

5.5 Dừng

docker compose -f docker-compose.single.yml down # giữ dữ liệu

```
PS D:\BIgdata_TroGiang\Config Hadoop\hadoop-docker> docker compose -f docker-compose.single.yml down
[+] Running 2/2
 ✔Container minhchau-master  Removed                                          2.5s
 ✔Network hadoopnet          Removed                                          0.5s
PS D:\BIgdata_TroGiang\Config Hadoop\hadoop-docker>
```

docker compose -f docker-compose.single.yml down -v # xóa luôn dữ liệu HDFS

---

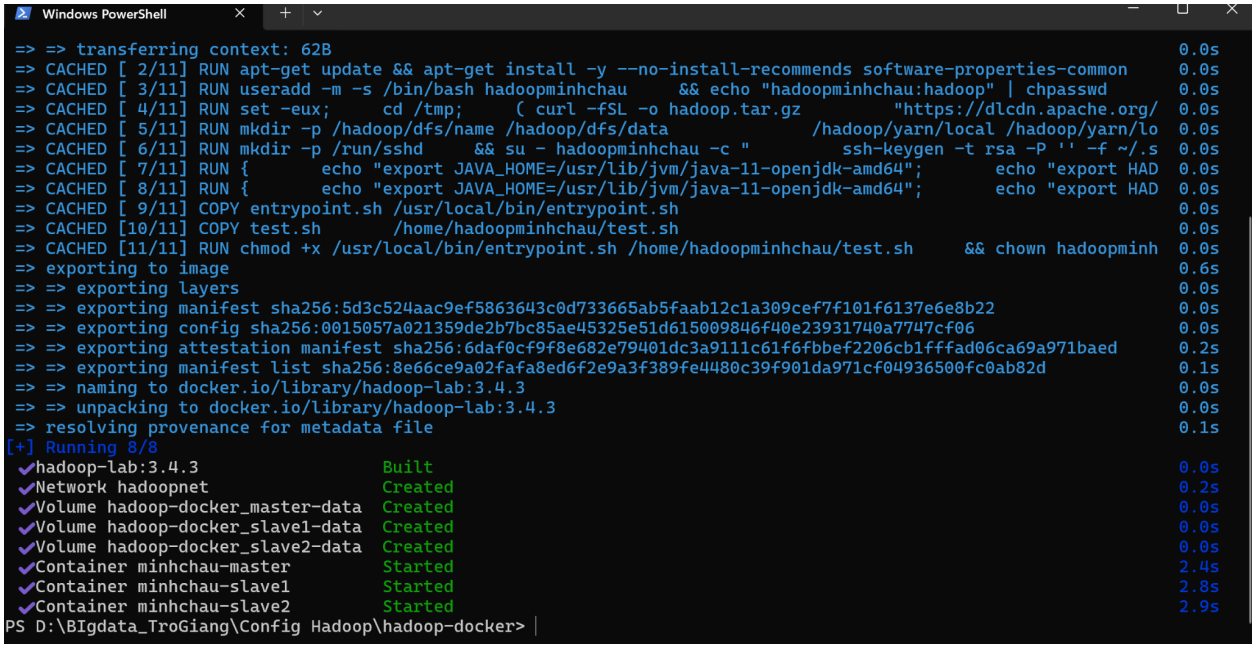
6. Phần B — Cluster 1 master + 2 slave

6.1 Khởi động

Nếu đang chạy Phần A thì tắt trước (vì 2 bên dùng chung tên container và cổng):

docker compose -f docker-compose.single.yml down

docker compose -f docker-compose.cluster.yml up -d --build



6.2 Kiểm tra

docker exec -u hadoopminhchau minhchau-master jps # NameNode, SecondaryNameNode, ResourceManager, JobHistoryServer

```
 ✔Container minhchau-slave2          Started                                                    2.9s
PS D:\BIgdata_TroGiang\Config Hadoop\hadoop-docker> docker exec -u hadoopminhchau minhchau-master jps
313 ResourceManager
186 SecondaryNameNode
110 NameNode
654 Jps
PS D:\BIgdata_TroGiang\Config Hadoop\hadoop-docker>
```

docker exec -u hadoopminhchau minhchau-slave1 jps # DataNode, NodeManager

```
654 Jps
PS D:\BIgdata_TroGiang\Config Hadoop\hadoop-docker> docker exec -u hadoopminhchau minhchau-slave1 jps
51 DataNode
115 NodeManager
329 Jps
PS D:\BIgdata_TroGiang\Config Hadoop\hadoop-docker>
```

docker exec -u hadoopminhchau minhchau-slave2 jps # DataNode, NodeManager

```
329 Jps
PS D:\BIgdata_TroGiang\Config Hadoop\hadoop-docker> docker exec -u hadoopminhchau minhchau-slave2 jps    # DataNode, Nod
eManager
114 NodeManager
51 DataNode
327 Jps
PS D:\BIgdata_TroGiang\Config Hadoop\hadoop-docker>
```

6.3 Kiểm tra slave còn sống

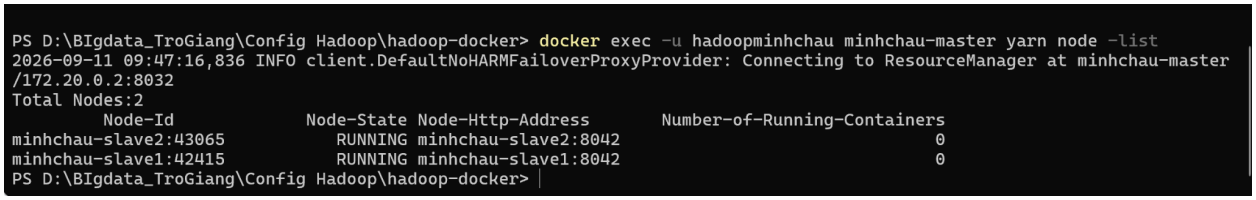
docker exec -u hadoopminhchau minhchau-master hdfs dfsadmin -report

```
327 Jps
PS D:\BIgdata_TroGiang\Config Hadoop\hadoop-docker> docker exec -u hadoopminhchau minhchau-master hdfs dfsadmin -report
Configured Capacity: 2162202353664 (1.97 TB)
Present Capacity: 2021334401024 (1.84 TB)
DFS Remaining: 2021334351872 (1.84 TB)
DFS Used: 49152 (48 KB)
DFS Used%: 0.00%
Replicated Blocks:
        Under replicated blocks: 0
        Blocks with corrupt replicas: 0
        Missing blocks: 0
        Missing blocks (with replication factor 1): 0
        Low redundancy blocks with highest priority to recover: 0
        Pending deletion blocks: 0
Erasure Coded Block Groups:
        Low redundancy block groups: 0
        Block groups with corrupt internal blocks: 0
        Missing block groups: 0
        Low redundancy blocks with highest priority to recover: 0
        Pending deletion blocks: 0

-------------------------------------------------
Live datanodes (2):

Name: 172.20.0.3:9866 (minhchau-slave1.hadoopnet)
Hostname: minhchau-slave1
Decommission Status : Normal
Configured Capacity: 1081101176832 (1006.85 GB)
DFS Used: 24576 (24 KB)
```

docker exec -u hadoopminhchau minhchau-master yarn node -list



`Live datanodes (2)` và 2 NodeManager ở trạng thái `RUNNING` là chính xác

---

7. Giải thích các file cấu hình

Hadoop đọc 4 file XML trong `$HADOOP_HOME/etc/hadoop/`. Quy tắc chung: **giá trị bạn không khai thì Hadoop lấy mặc định** trong các file `*-default.xml` nằm trong jar. Nên file cấu hình chỉ cần chứa phần bạn muốn khác mặc định.

core-site.xml — lõi (mọi node giống nhau)

Thuộc tính	Ý nghĩa
`fs.defaultFS`	`hdfs://minhchau-master:9000` — giúp xác định địa chỉ mặc định khi bạn gõ `hdfs dfs -ls /`. Đây là thứ nối mọi node về cùng một hệ thống file.
`io.file.buffer.size`	Buffer đọc/ghi, mặc định 4 KB. Tăng lên 128 KB giúp file lớn nhanh hơn.
`hadoop.tmp.dir`	Thư mục tạm gốc. Nhiều đường dẫn khác mặc định nằm dưới nó.

hdfs-site.xml — lưu trữ

Thuộc tính	Ý nghĩa
`dfs.replication`	Số bản sao mỗi block. **Không được lớn hơn số DataNode đang sống.** Single-node → 1, cluster 2 slave → 2.
`dfs.namenode.name.dir`	Nơi NameNode ghi metadata. Xóa thư mục này là mất toàn bộ cây thư mục HDFS.
`dfs.datanode.data.dir`	Nơi DataNode ghi block thật.
`dfs.blocksize`	Kích thước block, mặc định 128 MB. Tài liệu dùng 256 MB — block to thì ít metadata hơn nhưng song song kém hơn.
`dfs.namenode.handler.count`	Số luồng RPC của NameNode. Cluster nhỏ để 20 là thừa.
`*.bind-host = 0.0.0.0`	**Riêng cho Docker.** Bắt daemon lắng nghe mọi interface, nếu không bạn không mở được Web UI từ máy thật.
`dfs.client.use.datanode.hostname`	**Riêng cho Docker.** Cho client tìm DataNode bằng hostname thay vì IP nội bộ container.

yarn-site.xml — tính toán

Thuộc tính	Ý nghĩa
`yarn.resourcemanager.hostname`	Khai 1 dòng này là YARN tự suy ra cả 5 địa chỉ `:8030`–`:8033`, `:8088`. Tài liệu khai đủ 5 để bạn thấy từng cổng.
`yarn.nodemanager.aux-services`	Phải là `mapreduce_shuffle`. **Thiếu dòng này job MapReduce sẽ treo ở 0%.**
`yarn.nodemanager.resource.memory-mb`	Tổng RAM node này góp cho YARN. Tài liệu để 8192; trong Docker để 4096 hoặc thấp hơn tùy RAM bạn cấp cho Docker.
`yarn.scheduler.minimum/maximum-allocation-mb`	Giới hạn dưới/trên cho 1 container. Xin ít hơn min thì được làm tròn lên; xin quá max thì job bị từ chối.
`yarn.nodemanager.vmem-check-enabled = false`	**Riêng cho Docker.** JVM trong container thường bị tính dư bộ nhớ ảo → YARN giết container oan.
`...max-disk-utilization-per-disk-percentage = 99`	**Riêng cho Docker.** Mặc định 90%; đĩa Docker hay đầy hơn thế và node bị đánh dấu `UNHEALTHY`, không nhận job nào.
`yarn.nodemanager.env-whitelist`	Danh sách biến môi trường được truyền vào container của job.

mapred-site.xml — MapReduce

Thuộc tính	Ý nghĩa
`mapreduce.framework.name = yarn`	Chạy trên YARN thay vì chế độ local 1 tiến trình.

`yarn.app.mapreduce.am.env`
`mapreduce.map.env`
 	
`mapreduce.reduce.env`
	3 dòng tài liệu gốc thiếu. Không có chúng job chết ngay với `Could not find or load main class ...MRAppMaster`.

`mapreduce.map/reduce.memory.mb`	RAM cấp cho mỗi task.
`mapreduce.map/reduce.java.opts`	Heap của JVM bên trong task. Quy tắc: để **~75%** của `memory.mb`, phần dư dành cho bộ nhớ ngoài heap. Đặt `-Xmx` ≥ `memory.mb` là job bị YARN giết.
`mapreduce.jobhistory.*`	Địa chỉ và thư mục của JobHistoryServer.

workers

Danh sách hostname chạy DataNode + NodeManager, mỗi dòng một máy. **Chỉ `start-all.sh`/`stop-all.sh` trên master đọc file này.** Trong Docker, nó gần như chỉ có ý nghĩa tài liệu — vì mỗi container tự khởi động daemon của mình.

BÊN CẠNH ĐÓ, TÁC DỤNG CỦA TỪNG FILE ĐÃ GIẢI THÍCH RÕ Ở PHẦN 2, NẾU KHÔNG HIỂU Ý NGHĨA TỪNG THÀNH PHẦN THÌ ĐỌC LẠI Ở BÊN TRÊN CÓ GIẢI THÍCH TRONG KIẾN TRÚC

8. Xử lý lỗi thường gặp

`jps` không thấy DataNode Thường do NameNode đã bị format lại lần 2, khiến `clusterID` của NameNode khác của DataNode. Xem log `hadoop/logs/*datanode*.log` sẽ thấy `Incompatible clusterIDs`. Cách xử lý trong Docker:

docker compose -f docker-compose.cluster.yml down -v # xóa cả volume

docker compose -f docker-compose.cluster.yml up -d

Đây chính là điều mục 14 tài liệu cảnh báo: **`hdfs namenode -format` chỉ chạy một lần duy nhất.** `entrypoint.sh` đã tự bảo vệ — nó kiểm tra `/hadoop/dfs/name/current`, có rồi thì bỏ qua.

`Name node is in safe mode` NameNode vào chế độ chỉ đọc khi mới khởi động và chưa đủ tỉ lệ block báo cáo về. Chờ 1–2 phút, hoặc:

hdfs dfsadmin -safemode leave

Job treo ở `ACCEPTED`, không lên `RUNNING` Không có NodeManager nào nhận, hoặc node bị `UNHEALTHY`. Kiểm tra:

yarn node -list -all

Nếu thấy `UNHEALTHY` → thường là do đĩa đầy (đã xử lý bằng ngưỡng 99% trong `yarn-site.xml`) hoặc RAM cấp cho Docker quá ít. Giảm `yarn.nodemanager.resource.memory-mb` và các giá trị `mapreduce.*.memory.mb` xuống.

`Could not find or load main class ...MRAppMaster` Thiếu 3 dòng `*.env` trong `mapred-site.xml` — xem §10 mục 2.

`Connection refused` tới `minhchau-master:9000` NameNode chưa lên xong. Xem log:

docker compose -f docker-compose.cluster.yml logs master | tail -50

Web UI mở được nhưng bấm "Browse the file system" thì lỗi Trình duyệt trên máy thật đang cố nối thẳng tới DataNode. Cần `dfs.client.use.datanode.hostname = true` (đã có sẵn) và cổng 9864 phải được publish.

Build lâu / bị ngắt khi tải Hadoop `Dockerfile` thử `dlcdn.apache.org` trước, không được thì rơi về `archive.apache.org`. Nếu mạng chặn cả hai, tự tải `hadoop-3.4.3.tar.gz` về thư mục dự án rồi đổi bước tải trong `Dockerfile` thành `COPY hadoop-3.4.3.tar.gz /tmp/hadoop.tar.gz`.

Nguồn tham khảo

- Apache Hadoop — Release 3.4.3
- Apache Hadoop — Download
- Hadoop Java Versions (Apache cwiki)
- HADOOP-18887: Java 17 Runtime Support